Arithmetic with Python

COSC 1210 (CS I) Fall 26 | Augustana U.

Table of Contents

1. Objectives

Topics:

- Arithmethic operators
- Operator precedence
- Built-in functions
- Expressions
- String type
- Values and types
- Debugging

Source: Downey Think Python (3e)

2. Illusions of the machine

There are a number of illusions when you work with computing machines: You will understand them over time.

Illusion	Truth	Part
"The machine is one"	CPU + RAM + NVM	Computer Architecture
"Numbers are just numbers"	Numbers are not real	Representation
"The machine is all yours"	10,000 processes	Operating system
"The app is easy to use"	Know the interface	Application design
"You're the only one for me"	Many masters	Multi-tasking system

3. Arithmethic operators I

"Arithmetic" from Greek ἀριθμητική, the art of working with numbers (ἀριθμός).

Working with numbers requires ways of combining them. Another way to look at this is by imagining a space of numbers: Operators are a way of getting around that space from one number to another.

The basic ways of combining numbers are:

1. Addition
2. Subtraction
3. Multiplication
4. Division

4. Expressions

- Let's look at addition, subtraction, multiplication and division.
- An **expression** is any combination of numbers and these operators.
- There are only two operations here:
    1. Subtraction is addition with negative numbers: 2-3 is 2 + (-3)
    2. Division is multiplication with fractions: 10/2 is 10 * 1/2
- The machine exploits symmetries to reduce the number of operations:
    1. for addition/subtraction that's the symmetry around zero (0 is the identity element: a + 0 = a)
    2. for multiplication/division that's the symmetry around one (1 is the identity element: a * 1 = a)

- The machine cannot look at a formula as a whole, e.g. 29 + 4

```
print(29 + 4)
```

```
33
```

- Leaving out the `print` function for the time being (which you don't need in Jupyter if you only want to evaluate one expression), the machine evaluates from left to right.

- The machine is compelled to evaluate and reduce the expression to one value. Notice the special case of a unary operator (one argument):

```
print(-37)
```

```
-37
```

- Subtraction:

```
print(37 - 4)
```

```
33
```

Operator precedence

- When it encounters several operators and values, it applies operator precedence rules ("PEMDAS"):

```
print(3 * 5 - 5)    # from left to right, multiplication first: 15 - 5
print(5 - 5 * 3)    # from left to right, multiplication first: 5 - 15
print( (5 - 5) * 3) # from left to right, parentheses first: 0 * 3
print( 5 - (5 * 3))  # from left to right, 5 - 15
```

```
10
-10
0
-10
```

Division and floating-point numbers

- Division is different because it creates numbers with a decimal point, also called floating-point:

```
print(33/33)  # two integers divided result in a floating-point value
print(1/5)
```

```
1.0
0.2
```

Exponentiation and number representation

- Exponentiation is multiplication of the same number:

```
print(2**3)
print(2 * 2 * 2)
```

```
8
8
```

- Exponentiation can be tricky if the exponent is not an integer:

```
print(2**0.5) # the positive square root of 2; x**2-2=0 has two solutions
```

```
1.4142135623730951
```

- Digital numbers aren't really numbers:

```
print(f"{2**0.5:.2f}")  # rounded to 2 decimals after the point
```

```
1.41
```

- So which number is it?

  Neither. What's stored is an IEEE double (standard), good for about 15-17 significant decimal digits. Written out exactly, this particular double is:
  1.4142135623730951454746218587388284504413604736328125

- What `print` shows and why:

  The problem is that binary fractions (the way the computer stores them) do not cleanly map to decimal fractions. The `print` of a float shows the shortest decimal string that round-trips: the fewest digits that, when parsed back, land on that exact double. For this value, that's 1.4142135623730951 (17 digits).

- Code example:

```
import math
x = math.sqrt(2)
x
1.4142135623730951
print(float("1.4142135623730951") == x)  # now drop one digit
print(float("1.414213562373095") == x)   # different double
```

```
True
False
```

- Square root visualization:

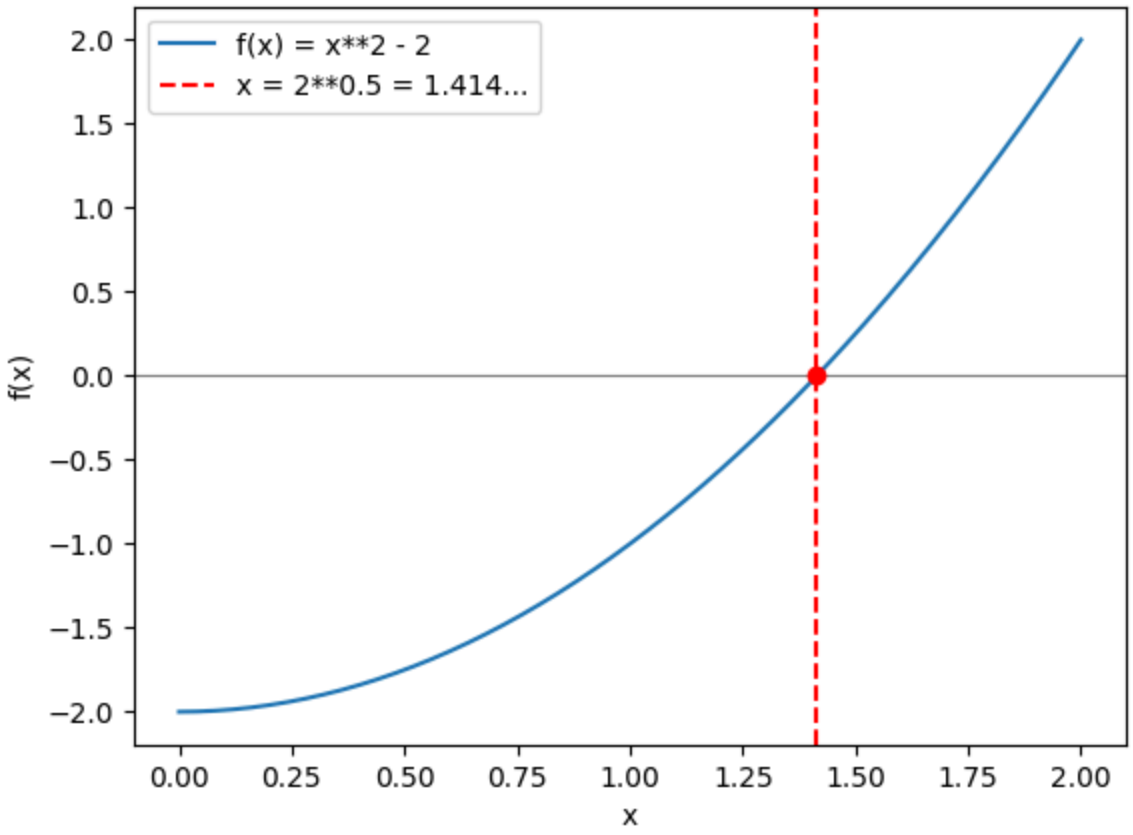
```
import matplotlib.pyplot as plt

def f(x):
    return x**2 - 2

# x and y values as list comprehensions
xs = [i / 100 for i in range(0,201)] # 0.00, 0.01...2.00
ys = [f(x) for x in xs]

# plot
plt.plot(xs,ys,label="f(x) = x**2 - 2")
plt.axhline(0,color="grey",linewidth=0.8) # horizontal line y = 0
plt.axvline(2**0.5,color="red",linestyle="--",
            label="x = 2**0.5 = 1.414...")
plt.plot(2**0.5, f(2**0.5),"ro") # the crossing point
plt.xlabel("x"); plt.ylabel("f(x)")
```

```
plt.legend()
plt.savefig("../img/squareRoot.png")
```



5. PRACTICE Formula translator

- Your turn! Compute the results of the following operation first **manually**, then write the code to confirm:

$$24 + 6/3 \times 5 \times 2^3 - 9 \qquad (1)$$

- Manually:

```
   24 + 6/3 * 5 * 2**3 - 9
-> 24 + 6/3 * 5 * 8    - 9
-> 24 + 6/3 * 40       - 9
-> 24 + 2 * 40         - 9
-> 24 + 80             - 9
```

```
-> 104                  - 9
-> 95
```

- Code check:

```
print(24 + 6/3 * 5 * 2**3 -9)
print(((6/3) * 5 * (2*2*2)) + 24 -9)
```

```
95.0
95.0
```

- Write the code for the following formulae:

$$10^2 + \frac{3 \times 60}{8} - 3 \tag{2}$$

$$\frac{5^3 \times (6-2)}{61-3+4} \tag{3}$$

$$2^{2+1} - 4 + 64^{-2^{2.25-\frac{1}{4}}} \tag{4}$$

$$\left(\frac{0.44 \times (1-0.44)}{34}\right)^{\frac{1}{2}} \tag{5}$$

- (2)

```
print(10**2 + ((3 * 60) / 8) - 3)
```

```
119.5
```

- (3)

```
print((5**3 * (6-2))/(61-3+4))
```

```
8.064516129032258
```

- (4)

```
print(2**(2+1)-4+64**((-2)**(2.25-(1/4))))
```

```
16777220.0
```

- (5)

```
print(((0.44 * (1 - 0.44)/34)**(1/2)))
```

```
0.08512965889470844
```

6. Arithmetic operators II

Bitwise operation `skip`

Bitwise operation: In some languages (like R), the caret `^` is an alternative exponentiation operator. In Python, it performs **bitwise XOR**:

```
print(7 ^ 2)
```

```
5
```

Bitwise operators treat numbers as if it were a string of bits, written in (two's complement) binary (base 2): Decimal `7` is equivalent to `111`, decimal `2` to `010`, and decimal 5 to `101`:

```
printf "%03d\n" "$(echo "obase=2; 7" | bc)"
printf "%03d\n" "$(echo "obase=2; 2" | bc)"
printf "%03d\n" "$(echo "obase=2; 5" | bc)"
```

```
111
010
101
```

The XOR operator (or exclusive OR) is 1 only if its arguments have different truth values (`1` for `true`, or `0` for `false`):

```
  111  (decimal 7)
^ 010  (decimal 2)
-----
  101  (decimal 5)
```

Bitwise operations are important in low-level (close to the machine) and performance-critical contexts: Systems programming, networking, and algorithms. Not needed for basic Python programming tasks.

More on bitwise operators and two's complement.

Integer division

Integer division, or **floor division**, returns the **floor** of the true result - the largest integer less than or equal to it:

```
print(85 // 2) # true numerical result: 42.5
```

```
42
```

Python uses `//` for floor division. This is a design choice - since `/` is already used for standard division, a related symbol was picked.

Modulo - division remainder

Modulo: This operator returns the remainder of a standard division:

```
print(10 % 2) # 10/2 = 5 : no remainder
print(11 % 2) # 11/2 = 10/2 + 1/2 = 5 + 1/2 : remainder 1
```

```
0
1
```

Notice that the space (aka **whitespace**) around the operators does not matter, as with all operators.

The modulo operator is used in many computing idioms - for example to identify even/odd numbers, for clocks/calendars, to split a total into units or drop the last digit (paired with integer division).

The operators `//` and `%` sit at the same precedence level as `*` and `/`, and are evaluated left to right.

7. Edge cases

- Doing computations on the computer (the main reason why it exists) is riddled with approximations.
- You don't always see this with the simple examples we've seen so far, but you see it when you try to compute so-called "edge cases", cases that are outside of what is expected most of the time.
- For example, if you multiply large enough numbers you can overflow to a special float value `inf` (try `1e308 * 10`). But there is no mathematical infinity here and no limit n -> ∞ as in math - `inf` just marks "too big to represent".
- Another one: what if you divide by zero - which in math yields "infinity" - what does the computer do with that? Is it allowed?

- Let's try the last case on the shell:

  ```
  python3 -c "print(1/0)" 2>&1
  ```

  ```
  Traceback (most recent call last):
    File "<string>", line 1, in <module>
  ZeroDivisionError: division by zero
  ```

- So **division by zero** leads to a `ZeroDivisionError` and aborts.
- Now you already know two error types: `SyntaxError` and `ZeroDivisionError`.

8. What's bash? Colab? Cloud shell? `skip`

- See what I just did there?

```
python3 -c "print(1/0)" 2>&1
```

- I am running Python on the terminal using the `bash` shell to directly execute it and pass a command as a string to the program `python3`. If that command generates an error, I am redirecting the error data stream (`2`) to the output data stream (`1`).
- Unfortunately, you cannot execute this command in JupyterLite because it is only an emulation of Python. Two other options:
    1. Open colab.research.google.com, then click on `Terminal`, then run the command. This is Google Colaboratory (as in the textbook).
    2. Open console.cloud.google.com, then run the command. This is a LinuxVM "in the cloud" created for Web developers.

9. PRACTICE Two simple programs

- Here are two simple scripts (another name for "Python program").
- Type the code into your Jupyter notebook.
- Can you tell what these scripts do? What one might use them for?

- First script:

```
print(153 // 60)
```

```
2
```

- Second script:

```
print(153 % 60)
```

```
33
```

Solution

- `153 // 60` computes how many 60s fit into 153. The answer is 2.
- `153 % 60` computes the remainder of `153 / 60`. The answer is 33.

- Together, these scripts can be used to convert minutes to hours and minutes (or seconds to minutes and seconds):

```
print( 153 // 60 ) # hours <- minutes
print( 153 % 60 )  # minutes <- minutes
print(f"153 minutes are {153//60} hours and {153%60} minutes.")
print(f"153 seconds are {153//60} minutes and {153%60} seconds.")
```

```
2
33
```

```
153 minutes are 2 hours and 33 minutes.
153 seconds are 2 minutes and 33 seconds.
```

10. PRACTICE Convert time units home

1. How many hours and minutes are in 222 minutes?
2. How many hours, minutes, and seconds are there in 10,000 seconds?

Solution

1. How many hours and minutes are in 222 minutes?

```
print(222 // 60) # hours <- minutes
print(222 % 60)  # minutes <- minutes
print(f"222 minutes is {222//60} hours and {222%60} minutes")
```

```
3
42
222 minutes is 3 hours and 42 minutes
```

2. How many hours, minutes, and seconds are there in 10,000 seconds?

```
print(10_000 // (60 * 60))  # hours <- seconds
print(10_000 % 3600 // 60)  # minutes <- seconds
print(10_000 % 60)  # seconds <- seconds

print(f"10,000 seconds are {10_000//3600} hours, {10_000%3600//60} minutes, and {1
```

```
2
46
40
10,000 seconds are 2 hours, 46 minutes, and 40 seconds
```

Notice that Python lets you make large numbers more readable by using underscores.

11. Arithmetic functions

- Besides operators, there are also arithmetic functions already available with Python without having to be loaded. Examples are `divmod`, `round` and `abs`.
- When a function is used like `print("hello")`, then `"hello"` is the **argument** (just like x in f(x)), and the function is said to be **called**.

12. Rounding with `round`

- `round` rounds a float to the nearest integer:

```
print(round(111.3))
print(round(111.7))
```

```
111
112
```

- What happens when the float is right in between two integers?

```
print(round(111.5))
```

```
112
```

- How can you find out what happens?

  In your browser, look for: `python doc round`. This gives you a list of all built-in functions in alphabetical order. You already know three of them: `print`, `divmod`, and `round`.

  Open the link. In the second paragraph you read that rounding between two equally distant integers is done towards the **even** choice. This round-half-to-even is also called "banker's rounding".

  You can test that with `round(3.5)` which should give `4` and not `3`.

```
print(round(3.5))
```

```
4
```

- Now look up the `help` on `round`:

```
help(round)
```

```
Help on built-in function round in module builtins:

round(number, ndigits=None)
    Round a number to a given precision in decimal digits.

    The return value is an integer if ndigits is omitted or None.  Otherwise
    the return value has the same type as the number.  ndigits may be negative.
```

- Notice how it does not give away the "banker's rounding"! `ndigits` is called a **parameter** of the function: parameters can be used to modify a function's behavior. For example, if you want to round to 2 digits ("the nearest hundredth"):

```
print(round(2.33))    # 0.33 < 0.5, rounds down to 2
print(round(2.333,2))  # sits between 2.33 and 2.34, 3 < 5, rounds down
print(round(2.677,2))  # sits between 2.67 and 2.68, 7 > 5, rounds up
print(round(2.675,2))  # sits between 2.67 and 2.68 exactly, rounds up?
```

```
2
2.33
```

```
2.68
2.67
```

- Why is `round(2.675,2)` not `2.68`?

  2.675 has no exact binary representation. The nearest double is 2.674999999999999822364316059974953532218933105468 75 < 2.675 so rounds **down**. The banker's rule is not consulted = it's not half-way.

- Takeaway:

  `round` always works from the **actual stored value**, not the decimal literal. A decimal ending in 5 is almost never a perfect tie in binary.

- Rounding is not truncating (cutting digits off).

13. Going absolute with `abs`

- The `abs` function returns the absolute value of a number:

```
abs(x) = x if x is greater or equal to zero
abs(x) = -x if x is smaller than zero
```

- Example:

```
print(abs(5))
print(abs(-5))
```

```
5
5
```

- When you look `abs` up in the documentation, you'll find this:

```
abs(number, /)
```

  The first parameter is `number`; the `/` is a marker meaning "everything before me is **positional-only**". This means that the argument `number` can never be passed with a name. The following gives an error:

```
python3 -c "abs(x=-5)" 2>&1
```

```
Traceback (most recent call last):
  File "<string>", line 1, in <module>
TypeError: abs() takes no keyword arguments
```

- This is our third error, a `TypeError`. We'll talk about **types** next.
- `abs` is a simple function to code by hand (you'll do it as an assignment in a couple of weeks).

14. String and "Hello, world!" in four languages

- A string is a sequence of characters like letters. Single or double quotation marks tell Python that it is working with a string.

```
print("Hello, world!")
```

```
Hello, world!
```

- Python has one of the shortest "Hello, world!" programs. By comparison, this is "Hello, world!" in Java:

```
public static void main(String[] args)
{
    String greeting = "Hello, world!";
    System.out.println(greeting);
}
```

- And this is "Hello, world!" in C++:

```
#include <iostream>

int main(int n, char* argv[])
{
  std::string s = "Hello, world!";
  std::cout << "Hello, world!" << std::endl;
}
```

```
Hello, world!
```

- But "Hello, world!" in R is even shorter, just like the name of the language:

```
"Hello, world!"
```

```
[1] "Hello, world!"
```

15. String operators

- The + and * operators don't "add" or "multiply" strings
- The + operator concatenates two strings to a single string:

```
print("Hello," + " world!")
```

```
Hello, world!
```

- The * operator makes multiple copies of a string and concatenates them:

```
print("Spam " * 5)
```

```
Spam Spam Spam Spam Spam
```

- What if you put the `5` outside of the `print` function? As in `print("Spam") * 5`?

```
python3 -c "print('Spam') * 5" 2>&1
```

```
Traceback (most recent call last):
  File "<string>", line 1, in <module>
TypeError: unsupported operand type(s) for *: 'NoneType' and 'int'
Spam
```

- Another `TypeError` - because you cannot multiply the integer `5` and the expression `print('Spam')`. You'll learn later why that is.

16. String functions

- Strings are actually quite a bit more complicated than "literals", objects that don't ever change, like numbers or letters - they're called "immutable": You cannot just change a letter in an existing string, you have to make a copy to do that.
- A String is what is called an Abstract Data Type (there's the **abstraction** again), or a `class`, something we made up to be able to handle strings as if they were numbers.

- When you create something like that, you have to define all kinds of operations (write them from scratch). E.g. for strings you can list these operations easily:

```
print(dir(str))
```

```
['__add__', '__class__', '__contains__', '__delattr__', '__dir__', '__doc__', '__e
```

- And if you only want to list the operations that you can use yourself:

```
[print(method) for method in dir(str) if not method.startswith('_')]
```

```
capitalize
casefold
center
count
encode
endswith
expandtabs
find
format
format_map
index
isalnum
isalpha
```

```
isascii
isdecimal
isdigit
isidentifier
islower
isnumeric
isprintable
isspace
istitle
isupper
join
ljust
lower
lstrip
maketrans
partition
removeprefix
removesuffix
replace
rfind
rindex
rjust
rpartition
rsplit
rstrip
split
splitlines
startswith
strip
swapcase
title
translate
upper
zfill
```

- Three easy functions from this list are self-explanatory:

    1. The length of a string:

    ```
    print(len("hello"))
    print(len("hello, world!"))
    ```

    ```
    5
    13
    ```

    2. Turning a string uppercase or lowercase:

    ```
    print("hello".upper())
    print("HELLO".lower())
    ```

    ```
    HELLO
    hello
    ```

- Notice that the `len()` function looks like a function with an argument `"hello"`, while the `upper()` and `lower()` functions don't have arguments at all, instead they get their input with a **dot-operator**.

17. PRACTICE Spaghetti with strings

- Work in pairs. The rule for this exercise: you may type each distinct word or symbol **once**. Every repetition has to come from the `*` operator.
- [10 minutes] Reproduce each of these strings with a single `print` statement, using only string literals, `+`, and `*`:
    1. `SpamSpamSpamSpamSpam`
    2. `Spam Spam Spam Spam Spam` (there is a space after every word, including the last one)
    3. `Spam, Spam, Spam, Spam, Spam, and Spam!`
- Before you run target 3, write down how many characters you expect the result to have. Then check with `len`.

- Stretch goal: print this sign using three `print` statements, one per line using the string `"spam"` in your code (not `"SPAM"`).

```
============
    SPAM
============
```

- Discuss with your partner:
    1. In target 3, does the `*` or the `+` happen first? How can you tell from the output?
    2. Try `print("Spam" * 5 + 2)`. Why does it fail?
    3. What does `("Spam, " * 5 + "and Spam!").upper()` give you? Is the pieced-together string still a "real" string? How could you tell?

Solution

- Target 1:

```
print("Spam" * 5)
```

```
SpamSpamSpamSpamSpam
```

- Target 2 - the trailing space is part of the literal:

```
print("Spam " * 5)
```

```
Spam Spam Spam Spam Spam
```

- Target 3 - `*` binds tighter than `+`, so `"Spam, " * 5` is built first and `"and Spam!"` is concatenated onto the end:

```
print("Spam, " * 5 + "and Spam!")
```

```
Spam, Spam, Spam, Spam, Spam, and Spam!
```

- Character count for target 3: `"Spam, "` is 6 characters, 6 * 5 = 30, plus 9 characters for `"and Spam!"` is 39:

```
print(len("Spam, " * 5 + "and Spam!"))
```

```
39
```

- Stretch goal:

```
print("=" * 12)
print(" " * 4 + "spam".upper())
print("=" * 12)
```

```
============
    SPAM
============
```

- Discussion: `"Spam" * 5 + 2` fails with a `TypeError` because `+` needs a string on both sides and `2` is an integer - the same reason `"Spam" + 5` failed earlier. And `.upper()` works on the whole concatenated string, because the result of `+` and `*` is just another string:

```
print(("Spam, " * 5 + "and Spam!").upper())
print(type(("Spam, " * 5 + "and Spam!").upper())) # is it still a string?
```

```
SPAM, SPAM, SPAM, SPAM, SPAM, AND SPAM!
<class 'str'>
```

18. Values and types

- One reason why people like Python: it does not force you to think about data types before using them. This is called "dynamic typing".
- A data type tells the machine how much memory to reserve. Even though it may seem to you that the machine's memory is huge, there are so many things to think of, so many resources to manage, that memory is actually quite precious.
- If you make mistakes with types, you get the `TypeError` that you already know.
- That's why other languages (Java, C/C++) spend a lot of energy on helping you help the machine to manage its memory resources.
- None of that in Python: it knows that `2` is an integer, `33.3` is a floating-point number, and `'Hello'` is a string.
- Lingo: numbers and words are **values**, and every value has a type - integer, string, float, or boolean (or truth type).

- The built-in function `type(object,/)` reveals the type for every value:

```
print(type(2))          # `int` is the integer number class
print(type(33.3))       # `float` is the floating-point number class
print(type("hello"))    # `str` is the string class
print(type(True))       # `bool` is the boolean value class
```

```
<class 'int'>
<class 'float'>
```

```
<class 'str'>
<class 'bool'>
```

19. Type conversion

- It turns out that you can convert types into other types, and sometimes you must do that (e.g. when you get data `input`):

  1. The built-in-function `int` converts to integer:

     ```
     print(int(33.3))
     ```

     ```
     33
     ```

     It does not know what to do with the fraction so it truncates towards zero:

     ```
     print(int(33.9))  # 33
     print(int(-33.9)) # -33 (closer to zero)
     ```

     ```
     33
     -33
     ```

  2. The built-in function `float` converts to float:

     ```
     print(float(33))
     ```

     ```
     33.0
     ```

- But what if you want to convert a string to an integer (or vice versa)?

  ```
  python3 -c  "print(int('hello'))" 2>&1
  ```

  ```
  Traceback (most recent call last):
    File "<string>", line 1, in <module>
  ValueError: invalid literal for int() with base 10: 'hello'
  ```

  This is a tasty new error, a `ValueError`

- This one on the other hand works better:

  ```
  print(int('444'))
  ```

  ```
  444
  ```

- What is the type of the result? Check it.

```
print(type(int('444')))
```

```
<class 'int'>
```

- Now the other way around: the built-in function `str` converts a number (or almost anything) into a string:

```
print(str(33))
print(str(3.14))
```

```
33
3.14
```

- The output **looks** the same, but the type has changed - check it:

```
print(type(str(33)))
```

```
<class 'str'>
```

- You need `str` whenever you want to join a number to a string with `+`. Without it, you get a `TypeError`:

```
python3 -c 'print("The answer is " + 42)' 2>&1
```

```
Traceback (most recent call last):
  File "<string>", line 1, in <module>
TypeError: can only concatenate str (not "int") to str
```

- Convert the number first:

```
print("The answer is " + str(42))
```

```
The answer is 42
```

20. Using `int` to convert binary to decimal `skip`

- Remember binary numbers? `int` is useful here, too. E.g. to convert binary `111` to decimal, you must tell the function that it is binary:

```
print(int('111',base=2))  # `base = 2` means binary
```

```
7
```

- The catch: you need to input the binary as a string, otherwise it won't work. Can you already predict what error you're going to get?

```
python3 -c "print(int(111, base = 2))" 2>&1
```

```
Traceback (most recent call last):
  File "<string>", line 1, in <module>
TypeError: int() can't convert non-string with explicit base
```

- Easy to remember: you used the wrong value -> `ValueError`. You used the wrong type -> `TypeError`.
- With what you learnt, can you now explain the error message from earlier for the command `int('hello')`?

```
ValueError: invalid literal for int() with base 10: 'hello'
```

Answer:

> Looking at the documentation, you see that the function is defined as `int(string, /, base=10)`. It expects the `string` to be convertible into a decimal number - like `444` earlier. But `'hello'` is not convertible into a decimal number. If you leave the `base` parameter out, the default is decimal. If you change it to `base=2`, you get a binary output.

21. PRACTICE Debugging

- Debugging has three aspects:
  1. Knowing your errors
  2. Finding your errors
  3. Fixing your errors
- Here is a short program with a bug. Copy it into pythontutor.com, step through it one line at a time, and watch the values appear in the panel on the right.

```
# Convert a Celsius temperature to Fahrenheit.
celsius = 25
factor = 9 / 5
fahrenheit = celsius * factor + 32
print(str(celsius) + " C = " + fahrenheit + " F")
```

Solution

- The first three lines run fine - the panel fills in `celsius = 25`, `factor = 1.8`, `fahrenheit = 77.0`.
- The `print` line stops with a `TypeError`: `can only concatenate str (not "float") to str`. You cannot join a string and a float with `+`; convert the number first:

```
# Convert a Celsius temperature to Fahrenheit.
celsius = 25
factor = 9 / 5
```

```
fahrenheit = celsius * factor + 32
print(str(celsius) + " C = " + str(fahrenheit) + " F")
```

```
25 C = 77.0 F
```

22. A gallery of errors

Python errors fall into three families: *syntax* errors (bad grammar — Python refuses to run), *runtime* errors (crash during execution), and *semantic* errors (runs, but gives the wrong answer — no message at all).

SyntaxError

```
# SyntaxError - Python cannot parse the code
print("hello)
```

```
  Cell In[1], line 2
    print("hello)
          ^
SyntaxError: unterminated string literal (detected at line 2)
```

Fix:

```
print("hello")
```

```
hello
```

IndentationError

```
# IndentationError - unexpected indent
print("hello", end=" ")
  print("world")
```

```
  Cell In[2], line 3
    print("world")
    ^
IndentationError: unexpected indent
```

Fix:

```
print("hello", end=" ")
print("world")
```

```
hello world
```

NameError

```
# NameError - variable is used before it was defined
print(age)
```

```
Traceback (most recent call last):
  Cell In[5], line 2
    print(age)
NameError: name 'age' is not defined
```

Fix:

```
age = 25
print(age)
```

```
25
```

TypeError

```
# TypeError - wrong type for the operation
print('3' + 3)
```

```
Traceback (most recent call last):
  Cell In[8], line 2
    print('3' + 3)
TypeError: can only concatenate str (not "int") to str
```

Fix:

```
print('3' + str(3)) # concatenating two strings
print(3 + 3)        # adding two integers
```

```
33
6
```

ValueError

```
# ValueError - right type, but the value cannot be converted
int("hello")
```

```
Traceback (most recent call last):
  Cell In[10], line 2
    int("hello")
ValueError: invalid literal for int() with base 10: 'hello'
```

Fix:

```
print(int("444"))
```

```
444
```

ZeroDivisionError

```
# ZeroDivisionError - dividing by zero is not defined
print(10/0)
```

```
Traceback (most recent call last):
  Cell In[12], line 2
    print(10/0)
ZeroDivisionError: division by zero
```

There's no fix but you can build guardrails around operations that might return this error, using the `try...except` command in Python.

ModuleNotFoundError

```
# ModuleNotFoundError - module does not exist or is not installed
import pandaemonium
```

```
Traceback (most recent call last):
  Cell In[13], line 2
    import pandaemonium
ModuleNotFoundError: No module named 'pandaemonium'
```

IndexError

```
# IndexError - index goes beyond the length of the list
x = [10, 20, 30]
print(len(x))   # notice that Python starts numbering at 0
```

```
3
```

```
print(x[3])
```

```
Traceback (most recent call last):
  Cell In[15], line 1
    print(x[3])
IndexError: list index out of range
```

Fix:

```
print(x[len(x)-1])
```

```
30
```

Semantic error

```
# Semantic error - runs, but gives the wrong answer
import math

radius = 5
area = 2 * math.pi * radius # wrong formula!

print(area)
```

```
31.41592653589793
```

Fix:

```
area = math.pi * radius**2
print(area)
```

```
78.53981633974483
```

23. A gallery of errors (CLI version)

Python errors fall into three families: *syntax* errors (bad grammar — Python refuses to run), *runtime* errors (crash during execution), and *semantic* errors (runs, but gives the wrong answer — no message at all).

Each block below runs independently so one error does not affect the next.

I am only using the command-line `python3` here so that you see the error displayed (otherwise it would only show on the console.

```
# SyntaxError — Python cannot parse the code
python3 -c "print('hello'" 2>&1
```

```
  File "<string>", line 1
    print('hello'
          ^
SyntaxError: '(' was never closed
```

```
# IndentationError — unexpected indent (a kind of SyntaxError)
python3 -c "
x = 1
  y = 2
" 2>&1
```

```
  File "<string>", line 3
    y = 2
IndentationError: unexpected indent
```

```
# NameError — variable used before it was defined
python3 -c "print(age)" 2>&1
```

```
Traceback (most recent call last):
  File "<string>", line 1, in <module>
NameError: name 'age' is not defined
```

```
# TypeError — wrong type for the operation
python3 -c "'3' + 3" 2>&1
```

```
Traceback (most recent call last):
  File "<string>", line 1, in <module>
TypeError: can only concatenate str (not "int") to str
```

```
# ValueError — right type, but the value makes no sense
python3 -c "int('hello')" 2>&1
```

```
Traceback (most recent call last):
  File "<string>", line 1, in <module>
ValueError: invalid literal for int() with base 10: 'hello'
```

```
# ZeroDivisionError — dividing by zero
python3 -c "10 / 0" 2>&1
```

```
Traceback (most recent call last):
  File "<string>", line 1, in <module>
ZeroDivisionError: division by zero
```

```
# ImportError — module does not exist
python3 -c "import pandamonium" 2>&1
```

```
Traceback (most recent call last):
  File "<string>", line 1, in <module>
ModuleNotFoundError: No module named 'pandamonium'
```

```
# IndexError — index beyond the list length
python3 -c "x = [1, 2, 3]; print(x[5])" 2>&1
```

```
Traceback (most recent call last):
  File "<string>", line 1, in <module>
```

```
IndexError: list index out of range
```

```
# Semantic error — runs, but gives the wrong answer (no message!)
python3 -c "
radius = 5
area = 2 * 3.14 * radius # wrong area formula: 2*PI*radius**2
print(area)              # prints 31.4 instead of 78.5
" 2>&1
```

```
31.400000000000002
```

- Syntax and indentation errors are caught *before* the code runs.
- Runtime errors crash the program *at the line* that fails.
- Semantic errors are the sneakiest: Python is happy, but you are not.

24. Summary

- An **expression** combines values and operators and always reduces to one value. Python reads it left to right, applying **operator precedence** (PEMDAS); `*`, `/`, `//` and `%` are all at the same level.
- Plain division `/` always produces a **float**. `//` gives the integer **floor**, `%` gives the remainder. Together, `//` and `%` split a total into units (minutes into hours-and-minutes, cents into coins).
- ** is **exponentiation**. A non-integer exponent (`2**0.5`) gives a float.
- Floats are IEEE doubles: about 15-17 significant digits, not exact. `print` shows the shortest decimal that round-trips to the stored value, so a literal "ending in 5" is almost never an exact tie.
- `round`, `abs` and `divmod` are **built-in functions** - no import needed. `round` uses **round-half-to-even**, and `round(x, n)` works from the stored value, so `round(2.675, 2)` is `2.67`.
- In a function signature, `/` marks **positional-only** parameters: the argument before it can never be passed by name.
- A **string** is characters in quotes. `+` concatenates strings, `*` repeats them; string **methods** (`upper`, `lower`, …) are reached with the dot-operator.
- Every **value** has a **type** - `int`, `float`, `str`, `bool`. Python is **dynamically typed**. `int()`, `float()` and `str()` convert between types - you need `str()` to join a number to a string with `+` - while `type()` reports a value's type. Wrong value -> `ValueError`; wrong type -> `TypeError`.
- Errors come in three families: **syntax** (caught before running), **runtime** (crash mid-execution, with a **traceback**), and **semantic** (runs, wrong answer, no message). Debugging is knowing, finding, and fixing them.

25. Glossary

These are the technical terms this one lecture touched - too many to memorise. You build a tech vocabulary by using the tech: in this case, by coding.

	Term	Definition
1	`abs()`	built-in: the absolute value of a number (its distance from zero).
2	argument	the value passed to a function when it is called.

	Term	Definition
3	banker's rounding	another name for round-half-to-even.
4	bitwise XOR (^)	combines two integers bit by bit; a bit is 1 where the inputs differ.
5	call	to run a function by naming it with parentheses, e.g. `round(2.5)`.
6	concatenation	joining strings end to end with `+`.
7	dynamic typing	Python assigns a value's type as the program runs, not in advance.
8	edge case	an input outside the usual range that breaks an assumption.
9	exception	an error raised while a program runs; it can be caught instead of crashing.
10	exponentiation (**)	repeated multiplication: `2**3` is `2*2*2`.
11	expression	any mix of values and operators that reduces to a single value.
12	float	a number with a decimal point, stored as an IEEE double (~15-17 digits).
13	`float()`	built-in: convert a value to a float.
14	floor division (`//`)	division rounded down to a whole number (the floor).
15	`inf`	a special float from overflow, meaning "too big to represent".
16	`int()`	built-in: convert to an integer (truncates toward zero).
17	modulo (`%`)	the remainder left after a division.
18	operand	a value that an operator acts on.
19	operator	a symbol that combines values: ~+ - * / // % **~.
20	operator precedence	the order operators apply when several appear (PEMDAS).
21	parameter	a name in a function's definition for an

	Term	Definition
		input it accepts.
22	positional-only (`/`)	a marker in a signature; parameters before it cannot be passed by name.
23	`round()`	built-in: round a float to a given number of decimal digits.
24	round-half-to-even	exact ties round to the even choice; Python's `round` rule.
25	runtime error	an error that stops the program while it is running.
26	script	another name for a program.
27	semantic error	the code runs but gives the wrong answer, with no error message.
28	string (`str`)	a sequence of characters written in quotes.
29	`str()`	built-in: convert a value to a string, e.g. to join a number to a string with `+`.
30	syntax error	code Python cannot parse; caught before anything runs.
31	traceback	the report Python prints when an uncaught error stops it.
32	truncation	cutting digits off (not rounding), e.g. `int(34.5)` converting float to integer
33	type	the kind of a value: `int`, `float`, `str`, `bool`.
34	`type()`	built-in: report the type of a value.
35	TypeError	using a value of the wrong type for an operation.
36	value	a piece of data; every value has a type.
37	ValueError	right type, but the value itself is unacceptable, e.g. `int('hello')`.
38	ZeroDivisionError	dividing by zero, which Python refuses.

Author: Marcus Birkenkrahe
Created: 2026-09-06 Sun 10:44